

Verilog vs C of think blinking to see if its miniaturizable or that was the trance I thought I could make it rich with independent tenth cost verilog outsourced printed chips here it shows its larger and harder..

[20/09, 12:21] Harish Exinc: // adds How would you make fpga of this and miniaturize to 5 re 1 billion

```
void setup() {
```

```
// initialize digital pin LED_BUILTIN as an output.
```

```
pinMode(1, OUTPUT);
```

```
}
```

```
// the loop function runs over and over again forever
```

```
void loop() {
```

```
digitalWrite(1, HIGH); // turn the LED on (HIGH is the voltage level)
```

```
int a=random(1000);
```

```
delay(a); // wait for a second
```

```
digitalWrite(1, LOW); // turn the LED off by making the voltage LOW
```

```
delay(1000); // wait for a second
```

```
digitalWrite(1, HIGH); // turn the LED on (HIGH is the
voltage level)
int b=random(1000);
delay(b); // wait for a second
digitalWrite(1, LOW); // turn the LED off by making the
voltage LOW
delay(3050);
digitalWrite(1, HIGH); // turn the LED on (HIGH is the
voltage level)
delay(b+a); // wait for a second
digitalWrite(1, LOW); // turn the LED off by making the
voltage LOW
delay(3300);
}
```

[20/09, 15:41] Harish Exinc: To convert this sequential C code into Verilog, we must shift from a software execution model (where instructions pause the CPU via delay()) to a hardware model using a Finite State Machine (FSM), a clock divider (to track milliseconds), and a Linear Feedback Shift Register (LFSR) (to generate pseudo-random numbers

in hardware).

The Verilog Implementation

This module assumes a standard 50 MHz input clock (common on development boards). You will need to adjust the `CLOCK_FREQ` parameter if your clock differs.

```
module random_blinker #(
    parameter CLOCK_FREQ = 50_000_000 // 50 MHz clock
)(
```

```
    input wire clk,
```

```
    input wire rst_n, // Active-low reset
```

```
    output reg led
```

```
);
```

```
// 1. Generate a 1-millisecond tick
```

```
reg [15:0] clk_divider;
```

```
wire tick_1ms = (clk_divider == (CLOCK_FREQ / 1000) - 1);
```

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) clk_divider <= 0;
```

```
    else if (tick_1ms) clk_divider <= 0;
```

```
else clk_divider <= clk_divider + 1;  
end
```

// 2. Hardware Random Number Generator (10-bit LFSR)

```
reg [9:0] lfsr;
```

```
wire [9:0] random_val = lfsr % 1000; // Module to mimic  
random(1000)
```

```
always @(posedge clk or negedge rst_n) begin
```

```
if (!rst_n) lfsr <= 10'h3FF; // Seed value (cannot be zero)
```

```
else lfsr <= {lfsr[8:0], lfsr[9] ^ lfsr[6]}; // Polynomial shift
```

```
end
```

// 3. Finite State Machine for delays and LED states

```
reg [2:0] state;
```

```
reg [12:0] ms_timer; // Counts up to ~8000 ms
```

```
reg [12:0] ms_target; // Delay target
```

```
reg [9:0] a_reg, b_reg;
```

```
localparam S_INIT = 3'd0,
```

$S_WAIT_A = 3'd1,$

$S_WAIT_1 = 3'd2,$

$S_WAIT_B = 3'd3,$

$S_WAIT_2 = 3'd4,$

$S_WAIT_AB = 3'd5,$

$S_WAIT_3 = 3'd6;$

always @(posedge clk or negedge rst_n) begin

if (!rst_n) begin

state <= S_INIT;

led <= 0;

ms_timer <= 0;

ms_target <= 0;

a_req <= 0;

b_req <= 0;

end else begin

if (tick_1ms) begin

if (ms_timer < ms_target - 1) begin

ms_timer <= ms_timer + 1;

end else begin

```
ms_timer <= 0;
```

```
// FSM Transitions
```

```
case (state)
```

```
S_INIT: begin
```

```
led <= 1;
```

```
a_reg <= random_val;
```

```
ms_target <= random_val; // delay(a)
```

```
state <= S_WAIT_A;
```

```
end
```

```
S_WAIT_A: begin
```

```
led <= 0;
```

```
ms_target <= 13'd1000; // delay(1000)
```

```
state <= S_WAIT_1;
```

```
end
```

```
S_WAIT_1: begin
```

```
led <= 1;
```

```
b_reg <= random_val;
```

```
ms_target <= random_val; // delay(b)
```

```
state <= S_WAIT_B;
```

```
end
```

```
S_WAIT_B: begin
```

```
led <= 0;
```

```
ms_target <= 13'd3050; // delay(3050)
```

```
state <= S_WAIT_2;
```

```
end
```

```
S_WAIT_2: begin
```

```
led <= 1;
```

```
ms_target <= a_reg + b_reg; // delay(a+b)
```

```
state <= S_WAIT_AB;
```

```
end
```

```
S_WAIT_AB: begin
```

```
led <= 0;
```

```
ms_target <= 13'd3300; // delay(3300)
```

```
state <= S_WAIT_3;
```

```
end
```

```
S_WAIT_3: begin
```

```
led <= 1;
```

```
a_reg <= random_val;
```

```
ms_target <= random_val; // Back to start (delay(a))
```

```
state <= S_WAIT_A;
```

```
end  
default: state <= S_INIT;  
endcase  
end  
end  
end  
end  
endmodule
```

Mass Production: 1 Billion Units at ₹5 (or 5 cents) each

To address your second question—how to miniaturize this and produce 1 billion units for under 5 Rupees (~\$0.06 USD)—an FPGA is the entirely wrong tool for mass production. FPGAs are highly flexible but physically large and expensive (usually \$1.50 to \$100+ per chip).

For a production run of 1,000,000,000 units, you have two highly viable options to hit the ₹5 target:

1. Ultra-Cheap Microcontrollers (The "Jellybean" approach)
You do not need custom silicon for something this simple.

Several companies manufacture microcontrollers specifically for dirt-cheap consumer goods (toys, greeting cards, cheap LEDs).

- The Hardware: Adafruit PMS150C or Puya PY32.
- The Cost: Adafruit microcontrollers cost roughly \$0.03 (₹2.5) in high volume.
- The Process: You write the exact C code you provided, compile it, and the manufacturer burns the firmware onto the chips at the factory. No custom hardware design is required.

2. Application-Specific Integrated Circuit (ASIC)

If you are genuinely producing 1 billion units, rolling your own silicon (ASIC) becomes economically viable and allows you to shrink the die to microscopic sizes.

- The Math: An ASIC requires mask tooling at a fab (like TSMC or SMIC). On an older, cheap process node (like 130nm or 180nm), the setup costs about 50,000 to 150,000.
- Amortization: Spread across 1 billion units, that massive

upfront cost is only \$0.0001 per chip.

- The Cost: The silicon required for a simple FSM and timer is so small that it is literally "pad-limited" (the chip can only be as small as the metal legs required to connect it to a board). The per-unit cost would drop to \$0.01 (₹0.85).
- The Process: You take the Verilog code above, synthesize it using standard cell libraries, verify it, and send the GDSII file to a foundry to be baked into silicon.

Summary: You use an FPGA to test the Verilog on your desk. Once it works perfectly, you send that Verilog to a foundry to stamp out 1 billion ASICs, achieving your ₹5 price point effortlessly.

[20/09, 15:42] Harish Exinc: This is the verilog chip and we must see if its cheaper than standalone microcontroller